A REPORT

ON

SWECHA PS-I PROJECT PORTFOLIO: TEAM PORTFOLIO, RAG CHAT BOT, ADHIKARSETU AND SMART STOCK DASHBOARD

BY

Name of the student	ID No.	Discipline
Soumyajit Rout	2024A7PS0002P	B.E. Computer Science Engineering

Prepared in partial fulfillment of the
Practice School – I

AT

Swecha

A Practice School – I

Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

June, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Practice School Division

Station:	Swecha
Duration:	May – July 2026
Date of start:	25 May 2026
Date of submission:	June 20, 2026
Title of the project:	HighOnCode Team Portfolio, Chat Bot, AdhikarSetu and Smart Stock Dashboard
Student(s) – ID, Name, Discipline:	2024A7PS0002P, Soumyajit Rout, B.E. in Computer Science Engineering
Expert(s):	Shashikanth P (Swecha Mentor), Ranjith Raj (Swecha Mentor), Rajashekhar (Swecha Mentor)
PS Faculty (FIC):	Dr. Tanmay Chatterjee
Key words:	Artificial Intelligence, Retrieval Augmented Generation, Natural Language Processing, Machine Learning, Stock Prediction, FastAPI, React, Next.js, Streamlit, Financial Analytics, Government Scheme Recommendation, Data Visualization, Portfolio Website, Frontend Engineering, Deployment
Project area(s):	AI/ML, NLP, Financial Analytics, Full-Stack Development, RAG Systems, and Data Visualization

Abstract:

This report presents the work carried out during Practice School-I at Swecha Telangana from May to June 2026. Four projects were completed, covering frontend and backend engineering, machine learning, natural language processing, retrieval-augmented generation, and deployment. Project 1, *HighOnCode Team Portfolio*, is an immersive server-side rendered portfolio website developed using React 19, TanStack Start, GSAP, and Framer Motion, and deployed on Cloudflare Workers. Project 2, *Document Intelligence Chatbot (ShipN-ChatBot)*, is a RAG-based conversational assistant integrating ChromaDB, Sentence Transformers, Groq Llama, Streamlit, and NLP analytics. Project 3, *AdhikarSetu*, is a hackathon-developed platform for MSME scheme discovery, where I implemented the complete frontend, developed three backend APIs, and managed deployment. Project 4, *Stock Intelligence Dashboard*, is a production-grade financial analytics platform incorporating machine learning models, FastAPI, React, scikit-learn, Supabase, and a multi-provider BYOK AI chat system. Collectively, these projects provided practical exposure to full-stack development, AI-driven applications, and scalable deployment practices.

Signature of student(s):	**Signature of PS Faculty (FIC):**
Date:	Date

Acknowledgements

I would like to express my deepest gratitude to my internship station, **Swecha**, for providing an outstanding technical environment and resources to work on cutting-edge full-stack and artificial intelligence applications during the Practice School-I program.
I am thankful to my industry mentors, **Shashikanth P**, **Ranjith Raj**, and **Rajashekhar**, for their consistent guidance, and architectural reviews that helped solve several environmental, hardware, and dependency bottlenecks during deployment.
I extend my sincere appreciation to my BITS faculty member, **Dr. Tanmay Chatterjee** , for his encouragement, and academic guidance throughout the duration of this program. Finally, I thank my team members and peers for their collaboration during the hackathon Projects.

Signature of the student

Contents

Chapter 1

Introduction

Practice School-I (PS-I) at Swecha Telangana, Hyderabad, provided an opportunity to apply classroom knowledge to real-world software engineering challenges. Over the course of May and June 2026, four projects were conceptualized, designed, developed, and deployed, spanning the domains of frontend engineering, retrieval-augmented generation, civic technology, and financial analytics.

This report is organized chronologically and presents each project in a self-contained section covering its objectives, architecture, methodology, results, and future scope. In addition, a dedicated section summarizes the technical learning and professional development gained through seminars, workshops, and self-directed study conducted during the PS-I period.

1.1 Overview of Projects

The four projects completed during PS-I are listed below.

1. **HighOnCode Team Portfolio** (May 2026) — An immersive, single-page SSR portfolio website for a three-member engineering collective, featuring a cinematic deep-ocean theme, GSAP and Framer Motion animations, and Cloudflare Workers deployment.
2. **Document Intelligence Chatbot (ShipNChat Bot)** (May 2026) — A RAG-based document assistant combining semantic search, conversational question answering, and automated NLP analytics over uploaded PDF documents.
3. **AdhikarSetu** (May–June 2026) — A group hackathon project providing MSME owners with government scheme discovery, benefit estimation, and rejection explanation through a full-stack web platform.
4. **Stock Intelligence Dashboard** (May–June 2026) — A production-ready, ML-driven stock analysis and prediction platform integrating scikit-learn ensemble models, a multi-provider BYOK AI chat system, and multilingual support.

Chapter 2

HighOnCode Team Portfolio

2.1 Project Overview and Objective

Project Title: HighOnCode Team Portfolio
Live URL: highoncode.vercel.app
Problem Statement: Conventional portfolio templates often fail to convey a team's technical depth, collaborative identity, and engineering culture.
Objective: To design and deploy a single-page portfolio for the three-member team using modern React frameworks and advanced frontend animations.

2.2 System Architecture and Tech Stack

The application was built using TanStack Start with React 19 and TypeScript, providing file-based routing and server-side rendering. Styling was implemented with Tailwind CSS v4, animations with GSAP and Framer Motion, smooth scrolling with Lenis, and deployment on Cloudflare Workers through the Nitro adapter. Since the project is entirely presentational, no database or backend API was required.

2.3 Methodology and Implementation

After initial AI-assisted scaffolding, the redesign, animation system, debugging, and deployment were completed manually. The website consists of Hero, Identity, Team, Skills, Learning, Workflow, Fun Zone, and Footer sections, with interactive dossier cards presenting member information. The interface adopts a deep-ocean theme featuring marine animations, sonar effects, wave dividers, a custom cursor, and adaptive motion profiles for accessibility and performance.

2.4 Results and Practical Applications

The portfolio delivers a distinctive online presence with fast loading and SEO advantages from SSR. The project strengthened expertise in React, TypeScript, GSAP, Framer Motion, and serverless deployment. Future work includes CMS integration, performance benchmarking, and GitHub API-based dynamic content.

Chapter 3

Document Intelligence Chatbot

3.1 Project Overview and Objective

Project Title: Document Intelligence Chatbot (ShipNChat Bot)
Live URL https://shipnchat.streamlit.app/
Problem Statement: Large PDF documents such as research papers, technical reports, and manuals are difficult to navigate manually. Users spend considerable time extracting relevant insights without any intelligent assistance.
Objective: To build a RAG-based document intelligence assistant that allows users to upload PDFs, query them through natural language, and receive automatically generated analytics including keyword extraction, entity recognition, sentiment analysis, readability assessment, and semantic clustering.

3.2 System Architecture and Tech Stack

Component	Technology
Embedding Model	SentenceTransformer all-MiniLM-L6-v2 (384-dim)
Vector Database	ChromaDB (local persistent, HNSW, cosine similarity)
Primary LLM	Groq llama-3.3-70b-versatile
Secondary LLMs	llama-3.1-8b-instant (Groq), llama3.2 (Ollama), OpenAI
Keyword Extraction	KeyBERT with Maximum Marginal Relevance
NLP	spaCy (NER), HuggingFace (sentiment), textstat (readability)
Visualization	Plotly
UI Framework	Streamlit

Table 3.1: Document Intelligence Chatbot – Technology Stack

3.3 Methodology and RAG Workflow

Figure 3.1 illustrates the end-to-end RAG pipeline.

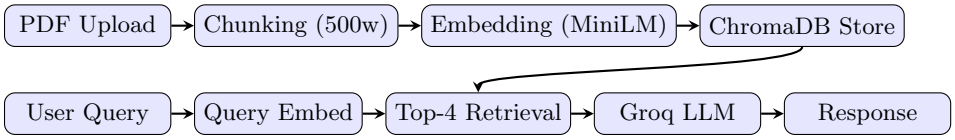

Figure 3.1: RAG Pipeline – Document Intelligence Chatbot

Document Chunking: Two strategies were implemented. The primary pipeline uses word-based chunks of 500 words with no overlap. A secondary implementation uses 160-word chunks with 35-word overlap for improved retrieval granularity. Chunks are embedded using `all-MiniLM-L6-v2`, generating 384-dimensional vectors stored in a locally persistent ChromaDB instance under `data/chroma_db/` using HNSW indexing with cosine similarity.
Retrieval and Generation: At query time, the user's question is embedded and the top-4 most similar chunks are retrieved using a cosine similarity threshold of 0.45. Retrieved chunks are injected into the prompt context before the Groq Llama 3.3 70B model generates a response.
KeyBERT: Keywords and keyphrases are extracted from the full document using Maximum Marginal Relevance to reduce redundancy and ensure diversity of extracted topics.

3.4 NLP Analytics Features

The Document Analysis tab provides four analytical sections, summarized in Table 3.2.

Feature	**Library**	**Output**
Keyword Extraction	KeyBERT (MMR)	Ranked keywords and phrases
Sentiment Analysis	HuggingFace	Polarity gauge, emotional tone charts
Named Entity Recognition	spaCy	Lists of persons, orgs, locations, dates
Readability Assessment	textstat	Flesch Reading Ease, complexity score
Semantic Clustering	scikit-learn (PCA + K-Means)	Interactive Plotly scatter plot
AI Insights	Groq LLM	Category, themes, writing style, suggested Qs

Table 3.2: Document Intelligence Chatbot – NLP Analytics

3.5 Results and Practical Applications

The system was validated on multiple PDF documents of varying lengths and domains. Top-4 retrieval with a 0.45 cosine threshold produced contextually relevant chunks across tested documents. Groq's inference API provided low-latency responses suitable for an interactive chat experience.
The platform has practical applications in academic research assistance, legal and compliance document review, enterprise knowledge management, and educational support. Future enhancements include multi-document querying, hybrid sparse-dense retrieval (BM25 + vector), and persistent user sessions with conversation memory.

Chapter 4

AdhikarSetu

4.1 Project Overview and Objective

Project Title: AdhikarSetu – AI-Powered MSME Benefit Discovery and Compliance Intelligence Platform
Live URL: adhikarsetu.vercel.app
Problem Statement: Micro, Small, and Medium Enterprises (MSMEs) in India often fail to access government welfare schemes due to complex eligibility criteria, bureaucratic language in rejection notices, and a lack of consolidated information. This results in significant unclaimed financial benefits.
Objective: To develop a web platform that recommends eligible government schemes to MSMEs, estimates potential unclaimed benefits, and simplifies rejection notices through a structured corrective action workflow. This was a three-member hackathon project developed over four days. My contributions are detailed explicitly in the sections below. The scheme matching engine was designed and implemented by a fellow team member and is not claimed as my work.

4.2 System Architecture and Tech Stack

Component	**Technology**
Frontend	Next.js 13, React 18, TypeScript, Tailwind CSS, shadcn/ui
Charts	Recharts
Forms & Validation	React Hook Form, Zod
Backend	FastAPI (Python 3.10+), Pydantic v2, Pandas, Uvicorn
Scheme Matching (teammate)	Rule-based eligibility engine (JSON datasets)
Deployment	Frontend: Vercel; Backend: Render
Version Control	GitHub, GitLab

Table 4.1: AdhikarSetu – Technology Stack

4.3 My Individual Contributions

Frontend Development: I designed and implemented the complete frontend. Pages developed by me include the landing page, user profile page, dashboard, benefit calculator interface, rejection explainer page, and policymaker dashboard. All Recharts visualizations, responsive layouts, animations, transitions, and API integrations were implemented by me. Local storage persistence for session state was also handled on the frontend.
Backend APIs (my implementation):

- *Loss Calculator API:* Accepts MSME profile inputs and estimates unclaimed government benefit value over a three-year horizon, producing annual distributions and cumulative projections.

- *Rejection Explainer API:* Analyzes uploaded rejection documents via keyword-based pattern matching and returns simplified plain-language explanations together with corrective action checklists.
- *Analytics Service:* Aggregates district-level participation records using Pandas to compute approval rates, exclusion rates, and regional awareness gaps for the policymaker dashboard.

Additional backend responsibilities I handled include file upload handling, validation layers, error handling, response schema design, data loading mechanisms, and API integration.

Deployment and DevOps: I deployed the frontend on Vercel and the backend on Render, configured production environments, and managed the project repositories on both GitHub and GitLab including branch management and debugging.

4.4 Platform Workflow

Figure 4.1 illustrates the user journey through the AdhikarSetu platform.

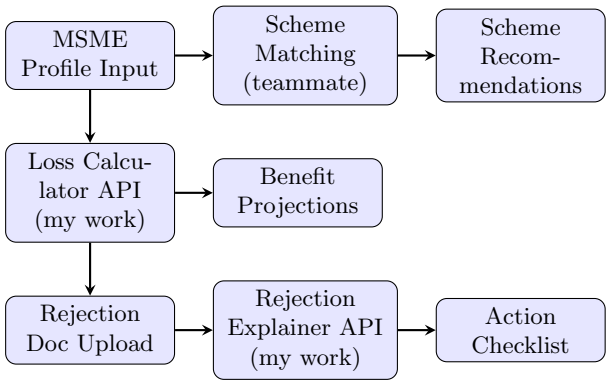

Figure 4.1: AdhikarSetu – User Workflow

4.5 Results and Practical Applications

The platform successfully demonstrated scheme discovery, financial benefit estimation, rejection analysis, and interactive analytics. It serves MSME owners seeking to identify relevant benefits, businesses aiming to understand rejection notices, policymakers analyzing participation trends, and government agencies identifying underserved regions. Future development will integrate PostgreSQL storage, machine learning-based eligibility scoring, and multilingual document processing.

Chapter 5

Stock Intelligence Dashboard

5.1 Project Overview and Objective

Project Title: Stock Intelligence Dashboard – AI-Powered Stock Analysis and Prediction Platform
Live URL: smart-stock18.vercel.app
Problem Statement: Retail investors rely on multiple disconnected tools for market analysis. Most platforms provide raw prices without intelligent analysis, and none integrate ML-based prediction, conversational AI, multilingual support, and interactive visualization in a single application.
Objective: To build a production-ready stock intelligence platform providing real-time data, ML ensemble price forecasting, AI-assisted research reports, BYOK multi-provider chat, and multilingual support. This was a fully individual project.

5.2 System Architecture and Tech Stack

Layer	**Technology**
Backend Framework	FastAPI (Python 3.11), Uvicorn
ML Library	scikit-learn (Linear Regression, Random Forest)
Data Source	Yahoo Finance (primary), Finnhub (company metadata)
Database	Supabase PostgreSQL
Frontend	React 18, TypeScript, Vite, Tailwind CSS
State / Data Fetching	Zustand, TanStack Query
Charts	Plotly
Internationalization	i18next (EN, HI, OD, DE, FR)
PDF Export	jsPDF + html2canvas
Deployment	Railway (backend), Vercel (frontend), Docker
CI/CD	GitLab Pipelines (lint, type-check, test, Docker build)
AI Providers	Groq, OpenAI, Anthropic, Gemini, OpenRouter, Ollama

Table 5.1: Stock Intelligence Dashboard – Technology Stack

5.3 Machine Learning Pipeline

Feature Engineering: The `FeatureEngineer` module computes the features listed in Table 5.2. RSI and EMA are additionally available for chart visualization.

Feature	Description
Close, Volume	Raw OHLCV inputs
MA7, MA21	7-day and 21-day moving averages
Returns	Percentage daily return
Lag1–Lag5	Previous 1–5 day closing prices
Volume Change	Percentage daily volume variation

Table 5.2: ML Feature Set

Prediction Target: Next trading day closing price ($y = \text{Close}_{t+1}$) using an 80:20 chronological train-test split. Models are cached as `ticker_range_model.pkl` via Joblib with LRU caching to manage Railway memory.

Ensemble Arbitration: Linear Regression and Random Forest produce independent forecasts. When both models predict the same trend direction, a weighted average proportional to confidence scores is used. On disagreement, the higher-confidence model's prediction is accepted and the other discarded. When the confidence difference is less than 0.05, the prediction is flagged as *Low Confidence*.

The confidence score is computed as:

$$\text{Confidence} = \frac{R^2 + 1}{2} \times (1 - \text{RMSE penalty}) \tag{5.1}$$

Performance metrics displayed to the user include RMSE, MAE, and R^2.

5.4 BYOK AI Chat and Report Generation

The platform implements a Bring-Your-Own-Key (BYOK) architecture. Users may enter their own API keys through the UI for any of the six supported providers (Groq, OpenAI, Anthropic, Gemini, OpenRouter, Ollama). Keys are stored exclusively in browser `localStorage` and are never persisted in Supabase or transmitted to the backend. A default Groq key is available as fallback.

Each AI conversation is injected with company profile data, historical prices, prediction metrics, and recent candle data as structured context before LLM inference. The platform also auto-generates an 11-section equity research report (executive summary through conclusion) exportable as PDF via jsPDF. Conversation histories are ticker-specific and persisted in `localStorage`.

5.5 Methodology Workflow

Figure 5.1 shows the end-to-end system workflow.

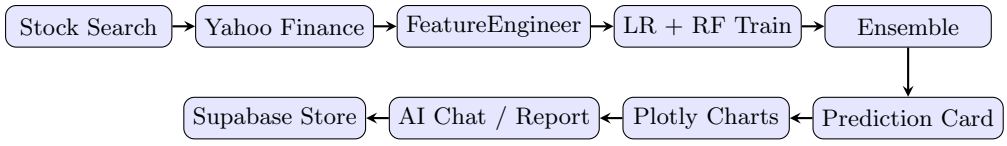

Figure 5.1: Stock Intelligence Dashboard – System Workflow

5.6 Database Design

Supabase PostgreSQL stores watchlists, search history, prediction records (predicted price, actual price, confidence, model name), and saved model metadata. Historical prices, chat history, and API keys are intentionally excluded from server-side storage for privacy and performance reasons.

5.7 Results and Practical Applications

The deployed platform successfully integrates stock data retrieval, ML prediction, AI-assisted analysis, and multilingual access. The ensemble arbitration mechanism provides more stable predictions than either model independently. The BYOK architecture makes the platform LLM-agnostic and cost-transparent. The GitLab CI/CD pipeline enforces code quality through formatting, linting, type-checking, security scanning, and coverage thresholds above 80%.
Future enhancements include directional accuracy tracking, multi-day prediction horizons, backtesting frameworks, portfolio-level analysis, and an authenticated user system with persistent preferences.

Conclusions

Practice School-I at Swecha Telangana provided valuable exposure to modern software engineering and artificial intelligence applications. The four projects undertaken during the internship spanned frontend engineering, retrieval-augmented generation, civic technology, and machine learning-driven financial analytics.

Each project involved end-to-end ownership, including requirement analysis, system design, implementation, testing, debugging, and deployment. These experiences strengthened practical knowledge of modern frameworks, cloud platforms, API design, vector databases, and machine learning workflows.

Project	**Primary Learning**
HighOnCode Portfolio	Advanced Frontend Engineering
ShipNChat Bot	RAG and NLP Analytics
AdhikarSetu	Full-Stack Development and APIs
Stock Dashboard	ML and AI Systems

Table 5.3: Summary of Technical Outcomes

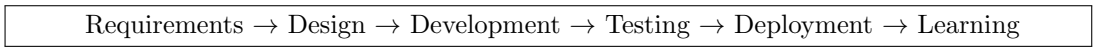

Figure 5.2: Software Engineering Workflow Followed During PS-I

The Stock Intelligence Dashboard demonstrated the integration of machine learning, conversational AI, and cloud-native technologies in a production-oriented setting. AdhikarSetu highlighted the challenges of building civic technology, while ShipNChat Bot provided hands-on experience with retrieval-augmented generation and NLP analytics. The HighOnCode Team Portfolio emphasized advanced frontend engineering and interactive design.

Overall, Practice School-I significantly enhanced technical proficiency, collaborative skills, and software engineering practices, providing a strong foundation for future academic and professional pursuits.

Bibliography

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Chroma. ChromaDB: The open-source embedding database. `https://www.trychroma.com`, 2023.

[3] Cloudflare. Cloudflare workers documentation. `https://developers.cloudflare.com/workers`, 2025.

[4] Docker Inc. Docker documentation. `https://docs.docker.com`, 2025.

[5] Framer. Framer motion documentation. `https://motion.dev`, 2025.

[6] GreenSock. Gsap documentation. `https://greensock.com/docs`, 2025.

[7] Maarten Grootendorst. Keybert: Minimal keyword extraction with bert. 2020.

[8] Groq. Groq api documentation. `https://console.groq.com/docs`, 2025.

[9] Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, 1986.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020.

[11] Meta Open Source. React – a JavaScript library for building user interfaces. `https://react.dev`, 2013.

[12] Fabian Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[13] Plotly Technologies Inc. Plotly python graphing library. `https://plotly.com`, 2025.

[14] Railway. Railway documentation. `https://docs.railway.com`, 2025.

[15] Sebastián Ramírez. FastAPI. `https://fastapi.tiangolo.com`, 2019.

[16] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[17] Streamlit. Streamlit documentation. `https://streamlit.io`, 2025.

[18] Supabase. Supabase documentation. `https://supabase.com/docs`, 2025.

[19] Tailwind Labs. Tailwind css documentation. `https://tailwindcss.com`, 2025.

[20] TanStack. Tanstack start documentation. `https://tanstack.com/start`, 2025.

[21] Vercel. Vercel documentation. `https://vercel.com/docs`, 2025.

Appendix A

Appendices

A.1 Live Project Links

- HighOnCode Team Portfolio: `https://highoncode.vercel.app`
- AdhikarSetu: `https://adhikarsetu.vercel.app`
- Smart Stock Dashboard: `https://smart-stock18.vercel.app`
- ShipNChat Bot: `https://shipnchat.streamlit.app/`

A.2 Technical Skills Acquired

- React 19, TypeScript, Next.js, TanStack Start
- FastAPI and REST API development
- Retrieval-Augmented Generation (RAG)
- Vector databases and semantic search using ChromaDB
- Machine learning with scikit-learn
- Data visualization with Plotly and Recharts
- Cloud deployment using Vercel, Railway, Render, and Cloudflare Workers
- Git, GitLab CI/CD, and Docker workflows

A.3 Learning Outcomes

The Practice School-I program strengthened skills in full-stack development, machine learning, prompt engineering, API design, cloud deployment, version control, debugging, and collaborative software engineering. The projects provided practical experience in designing and deploying production-grade AI-enabled applications.